

3D ULPIN Generation & Vertical Property Mapping System (VPMS)

Architecture & Technical Integration Analysis from God's Eye View (GEV) Repository

TECHNICAL SPECIFICATION

INTEROPERABLE 3D CADASTRE

Source Repo: [bilawalsidhu/gods-eye-view](#)
Target DB: **PostgreSQL 17** (46.247.108.191:30182)

Executive Technical Objective

Developing a scalable, ISO 19152 (LADM) compliant 3D Cadastral framework capable of generating standardized 3D ULPINs for **surface land parcels**, **multi-storey apartments**, and **underground infrastructure**. The system integrates Drone Imagery, LiDAR point clouds, GIS parcel layers, Building floor plans, GNSS/CORS coordinates, and Digital Elevation Models (DEM/DSM) with AI-assisted building extraction, floor segmentation, vertical parcel delineation, and intelligent topology validation.

Requirement in your 3D ULPIN Spec	GODS-EYE-VIEW (GEV) Source Module	Implementation Target in your Codebase	Key Benefit to your System
Surface Land Parcels (Authoritative Ground Cadastre)	<code>src/annotations</code> <code>src/hudLocality.js</code> <code>mgrs / pbf</code>	<code>LeafletMap.tsx</code> <code>parcels.py</code> <code>projection.py</code>	Establishes the authoritative 2D/3D ground cadastral base. Extrudes 2D GIS revenue polygons into 3D parcel volumes anchored to real terrain elevations with standard ULPIN geo-hashes.
Multi-Storey Apartments (Vertical Property Units)	<code>src/contactsDetectionPolicy.js</code> <code>src/hud.js</code> <code>src/splitFlap.js</code>	<code>FloorMesh.tsx</code> <code>BuildingMesh.tsx</code> <code>floors.py</code>	Wraps each individual apartment into a discrete 3D volumetric bounding box with Floor Level, Carpet Area (m²), Airspace Volume (m³), and its unique Vertical 3D ULPIN identifier.
Underground Infrastructure (Subsurface Utility Cadastre)	<code>src/layers/submarineCables</code> <code>src/scopeMask.js</code> (Depth-Masking Shaders)	<code>SubsurfaceAssets.tsx</code> <code>UndergroundMesh.tsx</code> <code>ulpin.py</code>	Enables depth-inverted X-ray viewing of buried utilities (gas, water, electric, metro) below the terrain without clipping; generates negative elevation Subsurface ULPINs (e.g. Z: -8.5m).
Drone Imagery Ingestion (Aerial Orthophotos & Viewshed)	<code>src/cockpitMath.js</code> <code>src/cameraVerbs.js</code> <code>src/layers/cctv</code>	<code>AddBuildingPipelineModal.tsx</code> <code>Scene3D.tsx</code>	Replays survey drone flight telemetry and projects 3D camera viewshed cones over the building to verify photogrammetric overlap (≥75%) for accurate 3D floor plan reconstruction.
LiDAR / Point Cloud Integration (Point Budgeting & Slicing)	<code>src/renderGovernor.js</code> <code>src/bloom.js</code>	<code>LidarPointCloud.tsx</code> <code>lidar_service.py</code>	Drapes high-density classified LiDAR point clouds directly over building models with selective point budgeting to prevent GPU lag; provides raw ground-truth height verification.
GIS Parcel Layers (Revenue Cadastral Synchrony)	<code>src/mapStackController.js</code> <code>src/layers/awareness</code> <code>@mapbox/vector-tile</code>	<code>LeafletMap.tsx</code> <code>Scene3D.tsx</code>	Connects local revenue vector boundary shapefiles/GeoJSON with 3D extrusions, ensuring exact synchrony between the 2D GIS cadastral registry and the 3D digital twin.
Building Floor Plans (2D CAD/DXF to 3D Volumes)	<code>src/annotations</code> <code>src/firstRunExperience.js</code> <code>src/locations.js</code>	<code>FloorMesh.tsx</code> <code>AddBuildingPipelineModal.tsx</code>	Ingests 2D architectural floor plans, extracts wall/room perimeters, and auto-stacks them at standardized vertical elevations (Z ₀ , Z ₁ , Z ₂ ...) as legal 3D spatial units.
GNSS/CORS Coordinates (Geodetic Datum Normalization)	<code>egm96-universal</code> <code>src/hudAltitudeDatum.test.mjs</code> <code>mgrs</code>	<code>coordinates.ts</code> <code>projection.py</code>	Harmonizes raw GNSS/CORS Ellipsoidal Heights (WGS84) with Orthometric Heights (Mean Sea Level via Geoid EGM96), ensuring accurate vertical datum benchmarks for 3D ULPINs.

Requirement in your 3D ULPIN Spec	GODS-EYE-VIEW (GEV) Source Module	Implementation Target in your Codebase	Key Benefit to your System
Digital Elevation Models (DEM/DSM) (Terrain Elevation Fusion)	src/maps src/mapStackChips.js (Cesium Ion / Esri Terrain)	TerrainMesh.tsx Scene3D.tsx	Displaces the 3D ground mesh using authoritative DEM/DSM elevation rasters so buildings, ground parcels, and road boundaries sit accurately on real topography without floating or sinking.
Automated Building Extraction (AI/ML Envelope Detection)	src/contactsDetectionPolicy.js src/camera.js	lidar_service.py BuildingMesh.tsx	Takes LiDAR clusters and drone imagery to calculate oriented bounding envelopes (OBB), generating the exterior volumetric shell of unregistered structures automatically.
Floor Segmentation (AI/ML Vertical Z-Slicing)	src/cockpitMath.js src/hud.js (Altitude Ladders)	floors.py FloorMesh.tsx	Analyzes building height profiles and LiDAR intensity breaks to automatically segment multi-storey towers into discrete horizontal floor slabs of standard heights (e.g. 3.0m).
Vertical Parcel Delineation (3D Volumetric Property Units)	src/annotations src/contactsDetectionPolicy.js	FloorMesh.tsx ulpin.py	Subdivides each floor slab into individual 3D property units (flats/offices), assigning each unit an explicit volumetric prism (X, Y, Z, ΔX, ΔY, ΔZ) and legal 3D spatial ID.
Intelligent Topology Validation (Overlap, Encroachment & FAR)	src/styles (FLIR / Thermal) src/bloom.js	Scene3D.tsx (Shaders) parcels.py	Uses false-color shader passes to flag spatial violations in real-time: highlights floor cantilevers projecting beyond ground boundaries (amber) and volumetric unit overlaps (red).
Standardized 3D ULPIN Generation (Surface, Vertical, Subsurface)	src/hudSummaryResponse.js mgrs / pbf	ulpin.py coordinates.ts	Formulates standardized alphanumeric 3D ULPINs integrating Geo-coordinates, Datum (+MSL), Elevation Range (Z_base → Z_top), and Sub-unit ID into an ISO 19152 (LADM) compliant code.
Vertical Ownership Rights (Landowner vs Unit Owner)	src/splitFlap.js src/hud.js	AddBuildingPipelineModal.tsx floors.py	Automatically enforces title assignment: unpurchased vertical units are automatically assigned to the base landowner, while sold units transfer title to individual apartment owners.
Underground Ownership & Easements (Subsurface Rights-of-Way)	src/layers/submarineCables src/annotations	SubsurfaceAssets.tsx parcels.py	Delineates municipal underground easements (e.g. 3m radius protective buffer around water pipes), preventing surface landowners from infringing on underground utility rights.
Volumetric Cadastre Systems (Common Property vs Private Rights)	src/annotations src/scopeMask.js	FloorMesh.tsx BuildingMesh.tsx	Explicitly demarcates Common Property Volumes (staircases, lifts, parking basements, roof terrace rights) separately from Private Volumetric Parcels in the database.
Urban Property Governance (AI Realtime Voice Assistant)	src/voice src/cameraVerbs.js	Header.tsx (Voice HUD) ulpin.py	Allows administrators to query records hands-free: "Show 3D ULPIN for Floor 4", "Highlight unauthorized constructions", or "Fly around parcel 204 boundary".
Ownership Dispute Reduction (3D Geodesic Whiteboard)	src/annotations (Geodesic Vector Whiteboard)	MeasurementTool.tsx Scene3D.tsx	Provides precision 3D dimension arrows (setbacks, party wall thickness, ceiling height, road margins) and saves timestamped 3D dispute markups to PostgreSQL for legal disputes.
Infrastructure Planning & Utilities (Multi-Layer GIS Context Switching)	src/contextModePolicy.js src/layers/traffic	LeafletMap.tsx SuperAdminDashboardModal.tsx	Enables city planners to overlay municipal utility plans, road widening reserves, and disaster hazard layers directly against registered 3D parcels.
Digital Property Certificates (Tamper-proof 3D Deed Links)	src/sharelink.js	ScannableQRCode.tsx	Serializes camera position, target volumetric parcel, elevation slice, and active layers into an instant 3D deep-link embedded into the property deed's scannable QR code.

Requirement in your 3D ULPIN Spec	GODS-EYE-VIEW (GEV) Source Module	Implementation Target in your Codebase	Key Benefit to your System
Scalable System Performance (Render Governor & Adaptive LOD)	src/renderGovernor.js src/lod.js	Scene3D.tsx TerrainMesh.tsx	Implements render-on-demand dirty checking (throttling idle frame rates) and adaptive Level-Of-Detail (2D boundary footprint → 3D building box → full interior floors) to render entire cities smoothly.